

Tackling Feature Selection Problems with *Genetic Algorithms* in Software Defect Prediction for Optimization

1st Rizal Broer Bahaweres

Department of Informatics
Syarif Hidayatullah State Islamic
University

Jakarta, Indonesia
Computer Science Dept.
IPB University

Bogor, Indonesia
rizalbroer@computer.org

4th Arif Imam Suroso

Computer Science Dept
School of Bussiness
IPB University

Bogor, Indonesia
arifimamsuroso@apps.ipb.ac.id

2nd Alam Wahyu Hutomo

Department of Informatics
Syarif Hidayatullah State Islamic
University

Jakarta, Indonesia
alam.wahyuh17@mhs.uinjkt.ac.id

5th Indra Permana Solihin

Computer Science Dept.
IPB University
Bogor, Indonesia
Informatics

Faculty of Computer Science.
Universitas Pembangunan Nasional
Veteran Jakarta
Jakarta, Indonesia
indrapermana@upnvj.ac.id

3rd Irman Hermadi

Computer Science Dept.
IPB University

Bogor, Indonesia
irmanhermadi@apps.ipb.ac.id

6th Yandra Arkeman

Computer Science Dept
Agro-Industrial Tech. Dept
IPB University

Bogor, Indonesia
yandra.arkeman@gmail.com

Abstract—Software defect prediction is a way to improve quality by finding and tracking defective modules in the software which helps reduce costs during the software testing process. The use of machine learning methods for predicting software defects can be applied to predict defects in each software module. However, basically the software defect prediction dataset has two problems, namely class imbalance with very few defective modules compared to non-defective modules and contains noisy attributes due to irrelevant features. With these two problems, it will result in overfitting and lead to biased classification results so that it will have an impact on significantly reducing the performance of the machine learning model. In this study, we propose the implementation of bagging techniques and genetic algorithms to improve the classification performance of machine learning models in predicting software defects based Logistic Regression, Naive Bayes, SVM, KNN, Decision Tree. Bagging techniques and Genetic algorithms are approaches that can handle two main problems in software defects prediction, each of which can handle the class imbalance and feature selection problem. We used 6 NASA Promise datasets to evaluate the classification performance results based on AUC and G-Means values. The results using 10 cross-validations show that the proposed method can improve classification performance when compared to the original algorithm. The Decision Tree shows the highest performance of the 3 datasets tested, with the highest value of 94.61% on the KC4 dataset. We also compare GA performance with another natural algorithm, Particle Swarm Optimization (PSO). The results show that the performance of all machine learning models with GA can outperform the algorithms with PSO

Keywords— *Software Defect Prediction, Feature Selection, Class Imbalance, Genetic Algorithm, Bagging, Particle Swarm Optimization*

I. INTRODUCTION

The structure of the software system is continues to develop so that it will basically become increasingly complex [1]. The more complex the software, the more costs required for the testing process [2]. In any area of software engineering, errors are sometimes inescapable and these errors mostly result into defects and failures in the software. Usually during the development process, software defects are discovered during software testing [3]. The software fault or defect occurs mainly due to the human operation while developing coding or due to the computer system programs [4].

Software defects are failures, errors, errors in program code that can reduce the quality of the software [5]. During software development, finding software defects is very expensive up to the software maintenance stage [3], so it will be complicated and time-consuming [6]. The quality of the software can be improved by minimizing the defects in the software.

Software defect prediction is a way to find and track defective modules in software [7]. Software defect prediction accuracy is needed to help reduce costs during the software testing process by focusing on defect-prone modules so as to improve prediction performance [5].

The classification algorithm is a machine learning approach that is useful for software defect prediction problems [8]. Basically, the software defect prediction dataset has two problems, namely class imbalance with very few defective modules compared to non-defective modules [9] and contains noisy attributes due to irrelevant features [10]. In the prediction of software defects, a machine learning model that is trained using a dataset that has class imbalance can cause overfitting which results in a decrease in the performance of

the machine learning model, because the majority of the class will be predicted as a non-defect [11]. Data sets that contain noisy attributes can affect data quality issues such as biased-unrepresentative data and missing or duplicate data and thus significantly reduce classification performance due to biased classification results [5]. The dataset commonly used in software defect prediction research is the public NASA PROMISE dataset [12][13][14]. The dataset has more defective data modules than non-defective modules [15] and has irrelevant features [10].

Feature selection is commonly used in the field of machine learning models to resolve datasets that have high dimensional or noisy attributes [10]. The use of feature selection can produce fewer features and thus reduce the computational costs of the model training process. Therefore, this data has to go through a preprocessing to ensure the data is cleaned out without noise and is also less redundant [7]. Most of the feature selection algorithms use local search in the whole process, so it is very difficult to obtain near-optimal to optimal solutions [5]. The genetic algorithm can find a variety of solutions to obtain a global optimal for feature search due to its ability to search for various regions in the solution space [16].

In this study, we propose the implementation of bagging techniques and genetic algorithms to improve the classification performance of machine learning models in predicting software defects based on Logistic Regression, Naive Bayes, SVM, KNN, Decision Tree. A Genetic algorithm is applied to handle the feature selection problem, and a bagging technique is applied to handle the class imbalance problem. Each classification algorithm is applied as a fitness value evaluator and classifier. We compared the results of classification performance with GA + Bagging and without GA + Bagging. We also compare GA + Bagging with another nature-inspired algorithm namely PSO + Bagging [5] as feature selection. As a performance evaluation of a machine learning model, we use the AUC metrics and G-means metrics to find the best machine learning model [17].

The research questions in this study are as follows:

1. How does the performance of software defect prediction classification with GA feature selection + Bagging compare without GA + Bagging ?
2. How does the performance of software defect prediction classification with GA + Bagging compare PSO + Bagging as feature selection ?

II. LITERATURE REVIEW

A. Related Research

TABLE I. RELATED WORK

Ref	Nasa Dataset	Bagging	Genetic FS	Other FS	Methods
[17]		✓			Bagging+Decision Tree
[5]	✓	✓		✓	PSO+Bagging+Many ML
[18]	✓			✓	BadSearch+Random Forest
[19]	✓		✓		Genetic+4 Neural Network
[20]			✓		Genetic+ANN
Author	✓	✓	✓	✓	Bagging+(Genetic+PSO)+Many ML

TABLE I describes research related to the study. In [17] applied of the ensemble technique using the Bagging algorithms and Decision Tree for classifier purpose. In terms of AUC and G-Means averages, the experimental findings indicate that the Bagging algorithm performs significantly better.

In [5] applied feature selection to 9 NASA MDP dataset using 11 classification algorithms. The implementation of bagging technique and particle swarm optimization is proposed for class imbalance problem and selecting the best feature. All machine learning models tested show that the use of PSO can improve the performance of the classification.

In [18] applied the Random Forest algorithm for the classification process based on feature selection optimization using the Bat-based Search algorithm. Of all the algorithms that have been tested based on the selection of the best features, it is found that the Random Forest algorithm shows the best results.

In [19] applied the proposed method where the features are selected using Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) and then train the model with different Neural Network (NN) methods such as: Feed forward Neural Network (FNN), Recurrent Neural Network (RNN), Artificial Neural Network (ANN), and Deep Neural Network (DNN). From this study, get the best accuracy result using deep neural network.

In this research [20] applied Genetic Algorithm feature selection to breast cancer diagnosis dataset. The classifier used is three different classifiers artificial neural network (ANN). The findings show that the feature selection will increase the specificity, precision and sensitivity of classifiers.

B. Feature Selection

In the classification process, all possible features will be used up for the training process, which can be encumbers for machine learning [21]. Feature selection is the process of selecting relevant features that are used prior to the classification process. The results of feature selection to improve the performance of the classification model results. Feature selection is also implemented to speed up the training process by reducing some uninformative features [22]. Usually, the feature selection process is used at the pre-processing data stage. The feature selection process that is carried out can reduce irrelevant features, so that reducing costs during the training process.

C. Bagging

Bagging is an ensemble method that is commonly used to improve classification accuracy by combining a single classification, and for class imbalance problems this method is better than the random sampling method. [23]. Therefore, to solve the class imbalance problem in the software defect prediction dataset, the bagging method is the solution to this problem.

D. Genetic Algorithm

Genetic Algorithm is an optimization method inspired by natural selection process [24]. The purpose of applied GA is to find the optimal solution in several sets of solutions. Each suite of solutions is referred to as a population. The population is formed based on a collection of chromosomes

or individuals. Each chromosome contains several genes [25]. In this study, GA is used to eliminate irrelevant features. We define chromosomes as a collection of features. In other words, each chromosome is a subset of features that have been selected. The size of the chromosome length (number of genes) is the same as the number of features contained in each NASA dataset. therefore, a chromosome is represented in the form of a binary string that is 1 or 0. the value 1 is the selected feature and 0 means it is not selected [20]. The general steps of GAs are summarized as follows [26] :

1. Generate an initial population randomly
2. Estimating each chromosome's fitness value in the population.
3. Perform the genetic operations, including the crossover, the mutation and the selection
4. If the termination condition is satisfied, end the algorithm; return to Step 2 otherwise. The termination condition is a predetermined number of generations.

After attribute selection is complete, the best attribute subset is processed to construct a classification model for performance evaluation (Figure 1). We use the accuracy of a classifier to evaluate the fitness size, the higher the accuracy rate, the greater the fitness. The Fitness was defined [27]:

$f(x)$ = AUC value. AUC value (min 0 - max 1) is defined by X-Axis and Y-Axis. It is an Area Under Curve. Y-Axis is represented by True Positive Rate or Sensitivity and X-Axis is represented by False Positive Rate or (1-Specificity).

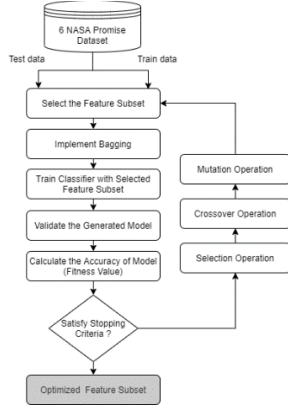


Fig. 1. Activity Diagram GA Feature Selection

E. Performance Evaluation

As an accuracy indicator to evaluate the performance of classifiers in our experiments we applied area under curve (AUC) and G-Means. The AUC is the best way to get general classifier performance values and compare them with other classifiers. The G-Mean are more comprehensive predictor evaluations in the context of class imbalance. The performance evaluation equation used to perform the calculations as given below [28][17][11] :

$$AUC = \frac{1 + TP_{rate} - FP_{rate}}{2}$$

$$G - Means = \sqrt{Sensitivity \times Specificity}$$

III. RESEARCH METHODOLOGY

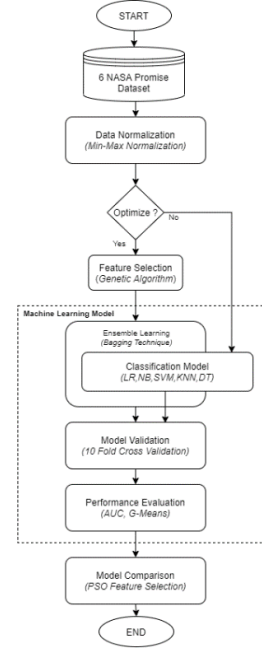


Fig. 2. Methodology

Fig 2 shows the methodology that will be used in this study. The first stage is collecting 6 NASA Promise datasets. After that, we applied the data normalization to each dataset. After being normalized, the features contained in each dataset will be selected using a genetic algorithm to find the optimal feature subset. We implement bagging techniques for each machine learning model by applying 10 cross validations to evaluate classification performance based on the selected feature subset. We also compare the performance of GA and PSO algorithm as feature selection. The performance evaluation that we will use is AUC and G-Mean. More explanation will be discussed in the next subsection.

A. Nasa Promise Dataset

In this study, The data sets used were taken from the public NASA Promise repository. This study used 6 data sets: CM1, KC1, KC2, PC1, PC2, PC4. The selected dataset is a dataset that has a class imbalance between a defective module with a not-defective module and also has a noisy attribute dataset. TABLE II is a list of datasets that will be used for research.

TABLE II. NASA PROMISE DATASET

Dataset	# Feature	# Instance	# Defective Instance (%)
CM 1	22	498	9.83
KC 1	22	2109	15.45
KC 2	22	522	20.50
PC 1	22	1109	6.94
PC 2	37	745	2.15
PC 4	38	1458	12.21

B. Data Normalization

The software metrics dataset has a high distribution variance. Data normalization is a technique commonly used in data preprocessing. The data normalization that we use is min-max normalization [29]. This technique aims to improve performance and speed up the classification process by changing the data distribution variance to between 0 and 1.

C. GA Feature Selection

After Normalization data, we divide the data to be applied to feature selection and without feature selection. We use

Genetic Algorithm as a feature selection, which has been mention in the literature review. It is observed that GA selects subset of features as chromosomes and each chromosome is evaluated by a classifier model to calculate the fitness value [20]. In the proposed method, chromosome is represented in the form of a binary string that is 1 or 0. the value 1 is the selected feature and 0 means it is not selected [25]. So that each gene on the chromosome determines which features accordingly should be used in classification model. The result of this selection is a subset of features that have a smaller number than the original feature. Eliminated features are irrelevant features. TABLE III show The parameter of selection, crossover and mutation used were applied using the following configuration :

TABLE III. PARAMETER SETUP

No	Parameter	Value
1	Population Size	50
2	Number of Generation	50
3	Chromosome Size	number of feature
4	Crossover Probability	0.5
5	Mutation Probability	0.2
6	Tournament Size	3

D. Model Validation

In dividing the dataset, we applied 10-fold Cross Validation technique. All data in the dataset has the opportunity to become training data and test data. We select a different part of the test dataset each time and use the remaining nine sections for learning. From ten separate test results, we then calculated an average value to determine the final result [5].

E. Classification and Performance Evaluation

After the training data set and test data set discard the unrelated feature subset and create new training data and test data. then, Classifiers are trained by a new training dataset based on the selected feature subset. Bagging was proposed to improve the classification by combining classifications of randomly generated training sets. The Bagging Classifier divides the training set into new training sheets using the random sample method, and builds a model based on the new training set. Then, the final classification results of several models were obtained by voting on each model. The classification model used was Naive Bayes, K-nearest neighbor, SVM, Logistic Regression, and Decision Tree. The hyperparameter of each algorithm that we use is the default scikit-learn. After carrying out the classification process, we calculate the average Area Under Curve (AUC) and G-Means value resulting from cross-validation. These results will be used to answer the research questions.

F. Comparison PSO Feature Selection

Particle swarm optimization is inspired by the social behavior of bird flocks and fish schools. In a dimensional discrete binary variable, a group of particles is randomly generated. The length of the particles is equal to the total number of features, and if the value of each particle one of the first i-bits is 1, then the first feature is selected, otherwise it will be protected. Each particle is a solution that becomes a candidate for the selection of subset features [5].

IV. RESULT AND DISCUSSION

We now answer our research question :

A. RQ1. How does the performance of software defect prediction classification with GA feature selection + Bagging compare without GA + Bagging ?

To answer this question, we tested and compared several machine learning model with/without GA Feature Selection and Bagging technique on 6 NASA Promise datasets in a case of Software defect prediction. We apply the Logistic Regression (LR), Naive Bayes (NB), K-Nearest Neighbour (KNN), Super Vector Machine (SVM) and Decision Tree (DT) algorithm. Testing is done using Python 3.6.9. The process of calculating Area Under Curve (AUC) and G-Means uses the 10-Fold Cross Validation method. The experimental result is shown in Table IV and Table V.

TABLE IV describes the results of the classification performance algorithm without GA + Bagging. Of the 4 datasets used, the Logistic Regression performance shows the highest performance among other algorithms based on the AUC value with the highest value of 85.29% in the KC2 dataset; On the other hand, Models based on the Decision Tree appear to be under performing for all datasets with the lowest AUC value of 48.97% on the PC2 dataset. In the PC2 dataset, the performance of 4 machine learning models shows a G-Means value of 0%, which indicates that the class imbalance impact on the dataset causes the classification algorithm to be unable to predict software defects.

TABLE IV. PERFORMANCE MODEL OF ALL ALGORITHM WITHOUT GA + B (%)

Classifier		CM1	KC1	KC2	PC1	PC2	PC4
LR	AUC	78.91	79.29	85.29	77.51	75.75	88.05
	G-Mean	14.25	40.74	48.94	16.09	0.	41.6
NB	AUC	76.21	78.82	84.63	70.8	76.68	84.54
	G-Mean	55.46	58.24	63.64	52.79	33.84	63.63
SVM	AUC	64.86	66.61	78.73	72.27	81.67	89.02
	G-Mean	0.	38.45	56.88	27.89	0.	38.92
KNN	AUC	68	72.86	78.62	75.55	55.89	84.88
	G-Mean	19.98	51.84	67.79	43.64	0.	56.05
DT	AUC	60.43	63.49	66.18	65.06	48.97	72.44
	G-Mean	52.03	59.08	63.12	54.43	0.	67.97

In the next experiment, we implemented GA and bagging technique for several machine learning model on 6 NASA Promise data sets. Bagging techniques will be applied to each machine learning model for training and testing based on a subset of features that have been optimized by genetic algorithms. Then the performance will be evaluated using 10 cross validation techniques. The experimental result is shown in TABLE V. Models that have improved performance for each classification algorithm are marked in bold.

TABLE V. PERFORMANCE MODEL OF ALL ALGORITHM WITH GA + B (%)

Classifier		CM1	KC1	KC2	PC1	PC2	PC4
LR GA+B	AUC	79.54	80.37	87.45	80.27	87.37	86.85
	G-Mean	65.23	67.38	74.82	61.17	81.46	78.7
NB GA+B	AUC	79.63	80.16	87.04	79.07	87.42	88.24
	G-Mean	57.89	58.77	71.15	56.63	81.77	78.19
SVM GA+B	AUC	80.2	80.01	86.78	82.14	87.89	89.97
	G-Mean	65.73	72.2	77.22	65.68	78.77	80.12
KNN GA+B	AUC	82.51	81.4	86.94	87.55	89.04	92.58
	G-Mean	72.58	72.22	78.66	78.45	79.38	84.72
DT GA+B	AUC	81.2	81.74	84.42	87.13	89.56	94.61
	G-Mean	71.14	72.75	74.49	80.25	84.15	89.12

Figure 3 visually shows AUC and G-Means value comparisons of 5 algorithms on 6 NASA Promise data sets. As shown in Table V and Figure 3, almost all machine learning models that implemented GA and bagging outperform the original method based AUC and G-Means value only the PC4 dataset the results of the performance of the LR algorithm with GA show an lowering in AUC value of 1.2%. This shows that the proposed method cannot solve the LR performance problem. Previously, the Decision Tree performance was the worst at original machine learning, now by applying the method proposed the Decision Tree algorithm is the best by outperforming the three datasets based AUC value with the highest value of 94.61% in the PC4 dataset. Conversely, the NB and SVM algorithms cannot outperform each dataset tested.

In summary, the combination of the Bagging Technique and GA-based feature selection is effective to improve classification performance significantly based AUC and G-Means value.

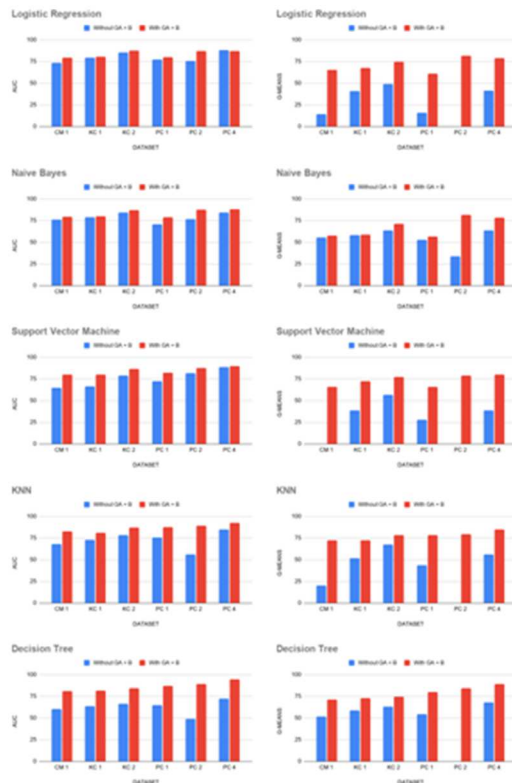


Fig. 3. Comparison of performance model all algorithm with/without GA+B

B. *RQ2. How does the performance of software defect prediction classification with GA + Bagging compare PSO + Bagging as feature selection ?*

To answer this question, we implemented bagging to compare GA and PSO as feature selection. The performance we tested was the same as before, Area Under Curve (AUC) and G-Means. The algorithm to be tested is the same as before, and we compare the results of the classification performance using GA and PSO as feature selection.

TABLE VI. PERFORMANCE MODEL OF ALL ALGORITHM WITH PSO + B (%)

Classifier		CM1	KC1	KC2	PC1	PC2	PC4
LR	AUC	77.5	79.15	85.32	76.23	78.13	85.18
PSO+B	G-Mean	64.04	69.87	77.94	58.92	71.69	75.15
NB	AUC	72.66	78.85	84.43	71.	76.23	84.86
PSO+B	G-Mean	54.27	60.78	67.79	54.29	67.47	71.06

Classifier		CM1	KC1	KC2	PC1	PC2	PC4
SVM	AUC	63.77	66.68	78.73	71.91	82.12	87.36
PSO+B	G-Mean	66.77	72.4	78.06	60.04	66.49	77.27
KNN	AUC	62.3	78.73	78.94	77.1	56.02	77.99
PSO+B	G-Mean	65.61	72.53	75.81	75.5	66.82	68.81
DT	AUC	79.25	78.45	82.02	82.08	84.27	88.69
GA+B	G-Mean	64.31	66.5	71.39	70.23	79.7	80.21

TABLE VI shows the results of the classification performance using a PSO feature selection. All algorithms with PSO gets less good results than GA feature selection based AUC value. However, there are several classification algorithms that have improved based on the G-Means value such as the SVM algorithm which outperformed the three datasets with the highest value of 78.06% on the KC2 dataset. In the KC1 dataset, there was an increase in the G-Means value for the three machine learning models tested. It indicates that the Bagging technique and PSO based feature selection has a significant effect on the performance of the classifier trained using a dataset that has a class imbalance. Models that have improved performance for each classification algorithm are marked in bold.

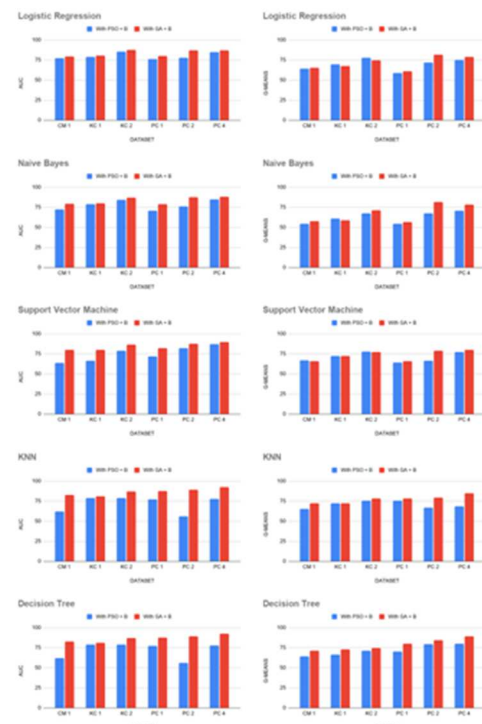


Fig. 4. Comparison of performance model all algorithm with GA+B and PSO+B

Figure 4 shows a graph of the comparison of the algorithm with GA and PSO. These results show that all algorithms with GA can outperform those with PSO. Both Area under curve (AUC) and G-Means value, GA are dominant. Feature selection with PSO results in lower performance than GA. However, the difference in results is not significant.

In summary, the feature selection with PSO can improve the performance of all classification algorithms but will get less good results when compared to the genetic algorithm feature selection.

V. CONCLUSION AND FUTURE WORK

A. Conclusion

In this study, we propose the combination of bagging techniques and genetic algorithms to improve the

classification performance of machine learning models in software defect prediction. The bagging technique was proposed to solve the class imbalance problem and the genetic algorithm was proposed for selecting the best subset of features. We performed comparisons of five machine learning models based on AUC and G-Means values applied to six NASA Promise datasets. The experimental results show that the proposed method achieved higher classification accuracy (AUC and G-Means) for almost all proposed machine learning models when compared to original machine learning. But on the PC4 dataset, the performance of the LR algorithm shows a decrease in the AUC value of 1.2% which indicates that the proposed method is not the answer to the LR algorithm performance problem.

We also compare GA with algorithms that are also inspired by nature, Particle Swarm Optimization(PSO). The experimental results show that all classification algorithms tested using GA show a better performance increase than PSO. However, it can be seen that the PSO method shows an increase in the G-means value for several machine learning models when compared to GA. Therefore, the PSO method can still be used as an option for feature selection because the performance is not significantly different from GA.

B. Future Work

For further research, we recommend that the optimization algorithm is not only applied to feature selection but can also be applied to optimize parameters in machine learning algorithms such as SVM, KNN, DT to improve software defect prediction performance

REFERENCES

- [1] R. Özakıncı and A. Tarhan, "Early software defect prediction: A systematic map and review," *J. Syst. Softw.*, vol. 144, pp. 216–239, 2018, doi: 10.1016/j.jss.2018.06.025.
- [2] W. P. K. N. V. C. K. W. B. Lawrence O. Hall, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, no. Sept. 28, pp. 321–357, 2002, [Online]. Available: <https://arxiv.org/pdf/1106.1813.pdf%0Ahttp://www.snopes.com/horror/insects/telamonias.asp>.
- [3] A. Balogun, "Software Defect Prediction : Effect of Feature Selection and," *Sci. Technol. J.*, vol. 3, pp. 518–522, 2018.
- [4] A. Rajasekaran and R. Varalakshmi, "An optimized feature selection using fuzzy mutual information based ant colony optimization for software defect prediction," *Int. J. Eng. Technol.*, vol. 7, no. 2.33 Special Issue 33, pp. 456–460, 2018, doi: 10.14419/ijet.v7i1.1.9954.
- [5] R. S. Wahono and N. Suryana, "Combining particle swarm optimization based feature selection and bagging technique for software defect prediction," *Int. J. Softw. Eng. its Appl.*, vol. 7, no. 5, pp. 153–166, 2013, doi: 10.14257/ijseia.2013.7.5.16.
- [6] P. A. Habibi, V. Amrizal, and R. B. Bahaweres, "Cross-Project Defect Prediction for Web Application Using Naive Bayes (Case Study: Petstore Web Application)," *2018 Int. Work. Big Data Inf. Secur. IWBIS 2018*, pp. 13–18, 2018, doi: 10.1109/IWBIS.2018.8471701.
- [7] M. Anbu and A. Mala, "Feature selection using firefly algorithm in software defect prediction," *Cluster Comput.*, vol. 22, pp. 10925–10934, 2017, doi: 10.1007/s10586-017-1235-3.
- [8] N. Babu, Himagiri, V. Vamshi Krishna, A. Anil Kumar, and M. Ravi, "Software defect prediction analysis by using machine learning algorithms," *Int. J. Recent Technol. Eng.*, vol. 8, no. 2 Special Issue 11, pp. 3544–3546, 2019, doi: 10.35940/ijrte.B1438.0982S1119.
- [9] Q. Song, Y. Guo, and M. Shepperd, "A Comprehensive Investigation of the Role of Imbalanced Learning for Software Defect Prediction," *IEEE Trans. Softw. Eng.*, vol. 45, no. 12, pp. 1253–1269, 2019, doi: 10.1109/TSE.2018.2836442.
- [10] Z. Xu, J. Liu, Z. Yang, G. An, and X. Jia, "The Impact of Feature Selection on Defect Prediction Performance: An Empirical Comparison," *Proc. - Int. Symp. Softw. Reliab. Eng. ISSRE*, pp. 309–320, 2016, doi: 10.1109/ISSRE.2016.13.
- [11] S. Wang and X. Yao, "Using class imbalance learning for software defect prediction," *IEEE Trans. Reliab.*, vol. 62, no. 2, pp. 434–443, 2013, doi: 10.1109/TR.2013.2259203.
- [12] L. Qiao, X. Li, Q. Umer, and P. Guo, "Deep learning based software defect prediction," *Neurocomputing*, vol. 385, pp. 100–110, 2020, doi: 10.1016/j.neucom.2019.11.067.
- [13] S. Huda et al., "An Ensemble Oversampling Model for Class Imbalance Problem in Software Defect Prediction," *IEEE Access*, vol. 6, pp. 24184–24195, 2018, doi: 10.1109/ACCESS.2018.2817572.
- [14] L. Gong, S. Jiang, and L. Jiang, "Tackling Class Imbalance Problem in Software Defect Prediction through Cluster-Based Over-Sampling with Filtering," *IEEE Access*, vol. 7, pp. 145725–145737, 2019, doi: 10.1109/ACCESS.2019.2945858.
- [15] J. Shirabad and T. Menzies, "The PROMISE Repository of Software Engineering Databases," 2005.
- [16] B. K. Khotimah, M. Miswanto, and H. Suprajitno, "Optimization of feature selection using genetic algorithm in naïve Bayes classification for incomplete data," *Int. J. Intell. Eng. Syst.*, vol. 13, no. 1, pp. 334–343, 2020, doi: 10.22266/ijies2020.0229.31.
- [17] H. Guan, Y. Zhang, H. Cheng, and X. Tang, "A novel imbalanced classification method based on decision tree and bagging," *Int. J. Performability Eng.*, vol. 14, no. 6, pp. 1140–1148, 2018, doi: 10.23940/ijpe.18.06.p5.11401148.
- [18] D. R. Ibrahim, R. Ghnemat, and A. Hudaib, "Software defect prediction using feature selection and random forest algorithm," *Proc. - 2017 Int. Conf. New Trends Comput. Sci. ICTCS 2017*, vol. 2018-Janua, pp. 252–257, 2017, doi: 10.1109/ICTCS.2017.39.
- [19] S. I. Ayon, "Neural Network based Software Defect Prediction using Genetic Algorithm and Particle Swarm Optimization," *1st Int. Conf. Adv. Sci. Eng. Robot. Technol. 2019, ICASERT 2019*, vol. 2019, no. Icasert, pp. 1–4, 2019, doi: 10.1109/ICASERT.2019.8934642.
- [20] S. Aalaei, H. Shahraki, A. Rowhanimanesh, and S. Eslami, "Feature selection using genetic algorithm for breast cancer diagnosis: Experiment on three different datasets," *Iran. J. Basic Med. Sci.*, vol. 19, no. 5, pp. 476–482, 2016, doi: 10.22038/ijbms.2016.6931.
- [21] U. Stanczyk, "Feature Evaluation by Filter, Wrapper, and Embedded Approaches," vol. 584, pp. 29–44, 2015, doi: 10.1007/978-3-662-45620-0.
- [22] S. B. Sakri, N. B. Abdul Rashid, and Z. Muhammad Zain, "Particle Swarm Optimization Feature Selection for Breast Cancer Recurrence Prediction," *IEEE Access*, vol. 6, pp. 29637–29647, 2018, doi: 10.1109/ACCESS.2018.2843443.
- [23] L. Hakim, B. Sartono, and A. Saefuddin, "Bagging Based Ensemble Classification Method on Imbalance Datasets," *IJCSN-International J. Comput. Sci. Netw.*, vol. 6, no. 6, pp. 670–676, 2017, [Online]. Available: <http://ijcsn.org/IJCSN-2017/6-6/Bagging-Based-Ensemble-Classification-Method-on-Imbalance-Datasets.pdf>.
- [24] A. K. Shukla, D. Tripathi, B. R. Reddy, and D. Chandramohan, "A study on metaheuristics approaches for gene selection in microarray data: algorithms, applications and open challenges," *Evol. Intell.*, no. 0123456789, 2019, doi: 10.1007/s12065-019-00306-6.
- [25] H. Vafaie and K. De Jong, "Genetic algorithms as a tool for feature selection in machine learning," *Proc. - Int. Conf. Tools with Artif. Intell. ICTAI*, vol. 1992-Novem, pp. 200–203, 1992, doi: 10.1109/TAI.1992.246402.
- [26] M. Dahane and L. Benyoucef, "A Literature Survey on Metaheuristics in Production Systems," vol. 60, no. August, pp. 93–107, 2016, doi: 10.1007/978-3-319-23350-5.
- [27] J. Murillo-Morera and M. Jenkins, "A software defect-proneness prediction framework: A new approach using genetic algorithms to generate learning schemes," *Proc. Int. Conf. Softw. Eng. Knowl. Eng. SEKE*, pp. 445–450, 2015, doi: 10.18293/SEKE2015-099.
- [28] I. H. Laradji, M. Alshayeb, and L. Ghouti, "Software defect prediction using ensemble learning on selected features," *Inf. Softw. Technol.*, vol. 58, no. September 2019, pp. 388–402, 2015, doi: 10.1016/j.infsof.2014.07.005.
- [29] S. G. K. Patro and K. K. sahu, "Normalization: A Preprocessing Stage," *Iarjset*, pp. 20–22, 2015, doi: 10.17148/iarjset.2015.2305.